FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int m, n, i, j;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], original[m];
    printf("Enter size of each block:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        original[i] = block[i];
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int process[n];
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int m, n, i, j;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], original[m];
    printf("Enter size of each block:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        original[i] = block[i];
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1